# SOLPS-ITER download and installation instructions

(please read ***in full*** before proceeding)

Welcome to SOLPS-ITER!

The instructions below are meant for ITER collaborators that have obtained an ITER IDM account in mind. Although parts of them also apply to users who obtain the code via the open-source ITER GitHub portal at https://github.com/iterorganization, they are still being updated to this new distribution channel. **The ITER Organization does not provide support to open-source users**, but they are welcome to report any issues they encounter and/or to start discussions about desired features at the respective locations on the ITER GitHub server.

In addition to the GitHub repositories, ITER IDM account holders may request access to other ITER repositories, which may not yet be publicly available. For that purpose, one must agree to the IMAS terms of use and fill out the form at: https://jira.iter.org/servicedesk/customer/portal/1/create/257. This may also require the signature of an IMAS Access Request between the IO and the new user's institution, if one is not already in force. Another option is for the user to be sponsored nominatively by a third party institution through their own IMAS Access Request. The only non-public IO repository related to SOLPS-ITER is AMNS ADAS, which provides data files that are not yet available on GitHub. For users without access to AMNS ADAS, the installation scripts will instead fetch equivalent data to that on the IO repository from the OpenADAS website (https://open.adas.ac.uk) and Zenodo. All other SOLPS-ITER-related IO repositories are only needed if you need to do some code archaeology, and no code development is to be done there. For IO repositories, if you do not intend to participate in code development, you can ask for REPO_READ permissions only, instead of REPO_WRITE. The open source repositories work on the fork system, so users may create their own forks, but the reference branches can only be manipulated by IO staff.

Users can obtain the EIRENE code under a CC-BY-NC-ND licence but developers need to agree to the terms of the Eirene Public Licence (EPL) and become a Basic Developer (BD) or Associated Developer (AD) - see *$SOLPSTOP/modules/Eirene/EPL.pdf* for full details. Such registration is managed by Forschungszentrum Jülich and occurs at https://www.eirene.de/. ADs associated to ITER may contribute by forking EIRENE and raising Pull Requests to have their changes integrated into the main repository.

You can generally browse the ITER Git repositories at https://github.com/iterorganization (open access) and https://git.iter.org (restricted access) and SOLPS-ITER specifically here: https://github.com/iterorganization/SOLPS-ITER. For the restricted ITER repositories, you will first need to authenticate with your ITER username and password – possibly twice, once for the intranet, and once for the Git web front end in the top right-hand corner.

If you upload a copy of your public *ssh* key at https://git.iter.org/plugins/servlet/ssh/account/keys or create a token at https://git.iter.org/plugins/servlet/access-tokens/manage, (by following links under “Manage Account”), then you should be able to simply clone a copy of SOLPS-ITER to your local machine by doing:

```
> git clone git@github.com:iterorganization/SOLPS-ITER.git
> cd solps-iter
> git submodule init
> git submodule update
```

If using the SSH method, make sure that there is no ‘PreferredAuthentications’ setting in your *ssh* configuration file, as it can conflict with the IO GIT repository access protocol. Please also note that the SSH key needs to be 1024 bits. For GitHub clones, be aware that the HTTPS method only allows

for read-only clones. If you intend to do code development, you must use the SSH cloning method and provide a public SSH key to GitHub, following the instructions you will find there.

If you want to clone SOLPS-ITER into a different directory than the “solps-iter” default name, replace the initial “git clone” command by:

> mkdir <my_personal_SOLPS-ITER-directory_name>
> cd <my_personal_SOLPS-ITER-directory_name>
> git clone git@github.com:iterorganization/SOLPS-ITER.git .

Please note that because of the way some of the SOLPS-ITER utility scripts function, the path to the SOLPS-ITER top directory must not contain any of the strings “.debug”, “.mpi”, “.openmp”, “.ig”, “.adj” or “.tgt”.

Since your user ID is immutably encoded into any changes, it's always a good idea to properly configure your local installation of Git on all the systems you plan on making code revisions on with, e.g.:

> git config --global user.name "Your Name"
> git config --global user.email your.name@where.you.are

It is recommended that you position yourself on the *master* branches (*SOLPS_master* for **Eirene**) in each of the submodules, by means of the *switch_to_master* script (which can be used after you establish your SOLPS-ITER session environment, see below), or by doing the following:

> git checkout master
> cd modules/B2.5
> git checkout master
> cd ../Eirene
> git checkout SOLPS_master
> cd ../DivGeo
> git checkout master
> cd ../Carre
> git checkout master
> cd ../adas
> git checkout master

If you are planning to do code development, your starting point should be the *develop* branch, which you can upload by using *solps-iter_update_develop*, after first checking out that branch at the SOLPS-ITER top level and rehashing.

If you wish to use the Wide Grids code version, then use the *solps-iter_update_wg_release* script to obtain the necessary branches.

When updating the code in this way, the script will check if you have any local branches where you have done some local work (or if your local work is not yet on a branch), and, if you have not done so already, ask you to commit or stash it first before proceeding. After the update is completed, you will be able to rebase your local work on top of the updated code as necessary. Please check the

*README* file in your *$SOLPSTOP* directory for further instructions on how to safeguard your work and update your code version.

The *solps-iter_update* scripts automatically recompile the code in standard mode (no debug, no MPI, no OpenMP) but will update the dependencies lists for all options. Not that, if the code update involves fetching new configuration files or changing code versions, the compilation triggered by the update script will, in all likelihood, fail. The procedure is then to close the current session, start a new one (which will then load the updated configuration files) and attempt a new compilation. If you wish to recompile with parallelization and/or debugging options, you can then do:

> gmake {your favourite target[s] here}

It is possible to pass *Makefile* instructions, in particular multi-threading over *nproc* processors, with either the syntax:

> gmake -j $(nproc) solps

Or, multi-threading for each individual SOLPS-ITER component, with

> gmake MAKE_OPTIONS=-j8 solps

The *Makefile* will internally parallelize all that is possible.

On GitHub or for ITER IDM account holders who have obtained REPO_WRITE permissions on ITER repositories, users can, for their own development, if any, create *$USERNAME/{feature/[topic]|bugfix/[topic]|hotfix/[topic]}* branches and raise a Pull Request (https://git.iter.org/projects/BND/repos/solps-iter/pull-requests) to have them merged into the develop branch after pushing them back to the ITER repository, where *$USERNAME* is their ITER IDM account name. This naming convention is meant to allow the SOLPS-ITER ROs to more easily manage everybody's branches. Please conform to it. The *develop*, *master*, and *svn/** branches are protected and can only be manipulated by the owners of the repository. For open-source users, please create a fork in which to make your own development, and then trigger a Pull Request using the relevant GitHub machinery.

Issues are handled using the "issues" and "discussions" tabs on the GitHub site. You should feel free to create new items associated with SOLPS-ITER or any of its submodules if you have any problems or suggestions. Use the "issues" tab to report bugs or unwanted code behaviour, and the "discussions" tab to propose new features and allow for debate about whether and how to implement them. The community of SOLPS-ITER users is expected to provide support to each other with the ITER Organization providing help as far as possible.

For help with Git, we can recommend the documentation here (http://git-scm.com/) and the Pro Git book (linked from there) in particular. Another useful resource for novice Git users is the tutorial from Codecademy available at https://www.codecademy.com/learn/learn-git.

ITER IDM account holders can find the documentation specific to SOLPS-ITER at:
https://iterorganization.sharepoint.com/sites/SOLPS-ITER/Shared%20Documents/General/
and it is visible from here:
https://imas.iter.org/ (then the "*Physics Components"* tab on the left, and choose *"SOLPS-ITER"* from the drop-down list, then follow the link to the official SOLPS-ITER documentation).

In addition, there is an IDM folder, located at https://user.iter.org/?uid=Q92BAQ , which contains links to many SOLPS-ITER related documents (those subject to document control procedures and/or presentations about aspects of the code to entities external to the IO, as well as related contract reports).

There is also a Confluence page at https://confluence.iter.org/display/IMP/SOLPS-ITER where contributions from users are welcome, as well as a Slack discussion group at https://solps.slack.com . Most of the contents from the Confluence and SharePoint pages can also be found on the SOLPS-ITER GitHub wiki (https://github.com/iterorganization/SOLPS-ITER/wiki), which users are of course welcome to further populate.

Lastly, the full documentation for use of the SOLPS-GUI can be consulted here:
https://static.iter.org/imas/assets/solps-iter/html/ .

Once you have cloned the SOLPS-ITER Repository, you may need to modify the configuration files needed so the necessary environment variables are properly defined. Please check the *README* file and the SOLPS-ITER User Manual (you can use the documentation link above) for details. These include your domain hostname, defined with the "whereami" script, your preferred compiler, in the "default_compiler" script, the modules you wish to load and your preferred debugger name (given with the *DBX=debugger_name* command) in *SETUP/setup.csh.$HOST_NAME.$COMPILER*, and the location and linking instructions for the various libraries in *SETUP/config.$HOST_NAME.$COMPILER*. Common instructions for a given compiler that are not host-dependent can be found in *SETUP/config.common.$COMPILER*. Feel free to look at the examples already present for inspiration. For hosts and compilers on which SOLPS-ITER has already been installed successfully and we know about, we have tried to include these files already in the distribution, but they may be mistaken or incomplete, so please check. In cases where the *HOST_NAME* is not correctly deduced from the "whereami" script, you can specify your desired *HOST_NAME* value by means of a *SETUP/setup.csh.HOST_NAME.local* file, which should contain the line:

setenv HOST_NAME <your_desired_HOST_NAME>

Alternatively, you can bypass the automatic *HOST_NAME* assignment by defining the environment variable *SOLPS_HOST_NAME_FORCE* in your current shell session. Similarly, you can override the automatic compiler setting by means of the definition of the environment variable *SOLPS_COMPILER_FORCE*. If you are installing SOLPS-ITER on a stand-alone laptop with a Linux operating system, you can specify "LINUX" for the *HOST_NAME* variable, or "DARWIN" if you are using a MacOS machine. For such stand-alone installations, only the *gfortran* toolchain is supported.

Once all variables are set, to start a SOLPS-ITER session (every time), from the SOLPS-ITER top directory, you do:

```
> tcsh
> source setup.csh
```

Then, if this is the first installation in your file space, please do:

```
> first_setup
```

This will, if they are not already present in the distribution, create the various configuration files for each of the SOLPS-ITER submodules. Feel free to modify them afterwards according to your local requirements. We strongly recommend that you do any modifications you may need in local copies of these files, to be named, respectively: *SETUP/setup.csh.$HOST_NAME.$COMPILER.local* and *SETUP/config.$HOST_NAME.$COMPILER.local*. Since these files are read in addition to the standard ones, they only need contain the settings required to be changed. Once you have made the proper modifications, if these are worth sharing with other users on similar platforms, please copy them other into the original files and commit them back to the repository using a pull request.

Then, if all was filled in properly, a simple "gmake" compilation command (and some patience) will get you a ready-to-run code. You should run the compilation from the SOLPS-ITER top directory, because this is where all the compiler flags are defined and passed down to each of the submodules. The top directory *Makefile* will also build the SOLPS-ITER Manual, which will be placed in *doc/solps/solps.[pdf|dvi|ps]*. The "gmake depend" command is necessary if you have not used one of the *solps-iter_update* scripts previously.

For the libraries required at compilation (see the full list in Section 1.2 of the SOLPS-ITER Manual), we recommend these be loaded as modules, which can then be shared across multiple users on a single machine, to make the SOLPS-ITER distribution lighter. With the exception of the GR, MSCL and libsonnet libraries, the libraries are not part of the SOLPS-ITER repository, as the IO does not have distribution rights for them. Libsonnet is built automatically from the SOLPS-ITER *Makefile* (although you may need to provide a machine architecture environment variable value). GR and MSCL can be obtained from GitHub at:

https://github.com/iterorganization/GR-Software
https://github.com/iterorganization/MSCL

When compiling the GR and MSCL libraries, you will need to set two environment variables, in the following way (provided you have already loaded the SOLPS-ITER environment as above):

```
> setenv OBJECTCODE $COMPILER
> setenv SOLPS_LIB $SOLPSLIB
```

Another available option for building the MSCL, GR, and GKS libraries, as well as some other publicly available ones for stand-alone installations, is for users to invoke the *install_dependencies* script which will download the library sources from their respective repositories, compile them automatically, and modify the local configuration files to provide the necessary linker instructions. This is the easiest and most automatic way of obtaining the necessary code dependencies.

Optionally, the entire suite of programs constituting the IMAS environment can be also locally installed, in a fully automatized way, by running the *install_imas* script. As a result, the IMAS components will be loaded upon entering the SOLPS-ITER environment, and full IMAS integration capabilities, such as reading/writing IMAS IDS, will be exploitable.

For the others, most are freely available from the Internet, with the exception of the NAG library, for which a pay licence is required. However, the NAG library is not strictly necessary to build SOLPS-ITER. And, if you already have an older version of SOLPS (5.x or 4.x), as distributed by IPP-Garching, you may copy the libraries from it into the *lib/$HOST_NAME.$COMPILER* directory (provided they are compiled with a compatible compiler version). Local installation of these libraries is the

responsibility of the users, and they should seek assistance from their local IT support staff, not from the IO which has neither the resources nor the capabilities to help in that respect beyond what is already provided in the distributed configuration files.

Another useful option available to build the `gfortran` and `ifort64` toolchains, including the IMAS modules and tools for SOLPS-ITER, is by means of the *SETUP/easybuild-local.sh* script. The script includes a complete list of modules as they appear on the ITER SDCC cluster and is convenient for standalone/local "builders" and for site administrators that want to build an ITER-equivalent installation on their machine. Example *setup.csh.UL.gfortran* and *setup.csh.UL.ifort64* are thus very similar to *setup.csh.ITER.gfortran* and *setup.csh.ITER.ifort64*, respectively. The configuration files *config.UL.gfortran* and *config.UL.ifort64* can then simply be symlinked to the corresponding *config.ITER* versions.

Essentially, this script exploits the functionality of the EasyBuild tool by adding search paths to ITER-specific easyconfig `.eb` sources from the ITER Git repository. The only requirement for this script is a recent version of Python 3 and `modulesfiles` or `Lmod` support. The rest is being downloaded from the internet and the ITER Git website. A quick start for building may be by creating a Personal Access Token from the ITER Git website as indicated above and entering:

```
> export EASYBUILD_MODULES_TOOL=Lmod # EnvironmentModules are default
> export HTTP_AUTH_BEARER="ReplaceWithPersonalAccessToken"
> SETUP/easybuild-local.sh --help | more
> SETUP/easybuild-local.sh
> SETUP/easybuild-local.sh --imas-foss install
> SETUP/easybuild-local.sh --imas-apps
```

Note that the above minimal example requires Python 3.8+, `Lmod` and Korn shell to be installed on the system and functional. The rest is installed by the script in the *easybuild.local* directory or elsewhere if desired.

If you have any questions about these instructions or encounter difficulties, feel free to contact the SOLPS-ITER RO at:
xavier.bonnin@iter.org